

第 01 章：点亮人生第一行代码 —— 串口通信与 Hello World

写在前面：如果你稍微懂一点编程（比如知道变量、循环、函数是什么），但对硬件、单片机和嵌入式开发完全是一头雾水，那么恭喜你，这本教程就是专门为你量身打造的！

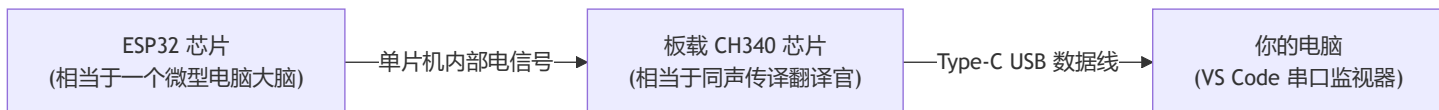
在电脑上学编程，第一课往往是在黑框框里打印一句 `Hello World`；而在单片机世界里，这一课同样是打印 `Hello World`，但意义却完全不同：它意味着你的开发板成功通电、大脑（CPU）开始思考，并且已经能够通过一条数据线与你的电脑进行“对话”了。

别害怕那些复杂的专业名词，让我们像拆积木一样，由浅入深一层一层搞懂它！

1.1 开发板与电脑是怎么“说话”的？

在开始写代码前，我们先搞清楚一个最基本的问题：**没有显示屏的时候，开发板怎么把数据显示到电脑屏幕上？**

答案就是：**串口通信（UART）**。



💡 形象比喻：两个人用“对讲机”聊天

- **波特率（115200）**：就像两个人对讲机调在同一个频道，并且**约定好说话的语速**。ESP32 以每秒 115200 个字的速度说，电脑也按这个速度听。如果两边速度对不上，电脑听到的就是“乱码天书”。
- **Type-C 数据线**：就是这根传话的线。（注意：必须是能传数据的数据线，不能是只能充电的纯充电线哦！）

1.2 关卡源码逐行带读（像读课文一样简单）

打开我们项目中的 `main/app_main.c`，代码一共只有 40 多行。我们把它切成 4 个小块来认识：

```
#include <stdio.h>
#include <inttypes.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"
#include "esp_system.h"
#include "esp_chip_info.h"
#include "esp_flash.h"
```

👉 第一部分：工具箱借工具（引入头文件）

- `#include` 的作用就是“从系统和框架里借工具”；
- 想打印文字？借 `stdio.h` 和 `esp_log.h`；

- 想让程序休眠延时？借 freertos/task.h ；
- 想查查这块芯片有几个核心？借 esp_chip_info.h 。

```
static const char *TAG = "LEVEL_1_HELLO";
```

👉 第二部分：给自己的模块起个名字（TAG）

- TAG 就是一个字符串标签（这里叫 "LEVEL_1_HELLO" ）；
- 以后打印的所有日志前面都会自动带上这个名字，方便在成千上万行日志里一眼找到“是谁说的话”。

```
void app_main(void)
{
    ESP_LOGI(TAG, "=====");
    ESP_LOGI(TAG, "      🎉 恭喜! ESP32 关卡 1 启动成功!      ");
    ESP_LOGI(TAG, "=====");

    esp_chip_info_t chip_info;
    uint32_t flash_size;
    esp_chip_info(&chip_info);

    ESP_LOGI(TAG, "【硬件信息】CPU 核心数: %d 核", chip_info.cores);
    ESP_LOGI(TAG, "【硬件信息】芯片版本: v%d", chip_info.revision);

    if (esp_flash_get_size(NULL, &flash_size) == ESP_OK) {
        ESP_LOGI(TAG, "【硬件信息】板载 Flash 大小: %" PRIu32 " MB", flash_size / (1024 * 1024));
    }
}
```

👉 第三部分：开机自检与亮明身份

- app_main 是 ESP32 启动后**第一个执行的入口函数**（相当于单片机的开机起点）；
- esp_chip_info：单片机自己摸摸自己的脑袋，看看自己是单核还是双核；
- esp_flash_get_size：看看自己身上带了多大的“硬盘”（Flash 容量）。

```
int count = 0;

while (1) {
    count++;
    uint32_t free_heap = esp_get_free_heap_size();

    ESP_LOGI(TAG, "[%04d] Hello ESP32! 当前空闲内存: %" PRIu32 " 字节", count, free_heap);

    printf("      -> 来自 printf 的问候: 距离开机运行已过去 %d 秒\n", count);

    vTaskDelay(pdMS_TO_TICKS(1000));
}
}
```

👉 第四部分：永远不休息的心跳（主循环）

- 变量 `count` 记录开机跑了多少秒；
- `esp_get_free_heap_size()`：看看当前单片机还剩多少运行内存；
- `vTaskDelay(...)`：睡上 1 秒钟，然后醒来继续循环。

1.3 核心疑问解答（小白必问的 4 个为什么）

疑问 1：为什么单片机程序必须要写 `while (1)` 死循环？

- **电脑软件**：比如一个计算器程序，算完 $1 + 1 = 2$ 之后，程序退出，窗口关闭，非常正常。
- **单片机设备**：想想你家里的微波炉、电风扇或者智能手表。通电开机之后，它绝不能“跑完代码就退出关机”，它必须 **7×24 小时活着，时刻准备响应按键、检测传感器**。
- 这个 `while (1)` 就是让单片机“永动机”般持续运转的核心机制。

疑问 2：为什么不用 `printf`，而是推荐用 `ESP_LOGI`？

在电脑上我们习惯写 `printf("hello\n")`，但在单片机开发中，官方更推荐 `ESP_LOGI`。为什么呢？

对比一下它们的输出效果你就懂了：

裸 `printf` 输出：
`hello, count = 1`

`ESP_LOGI` 输出：
`I (1360) LEVEL_1_HELLO: [#0001] Hello ESP32! 当前空闲内存: 298412 字节`

`ESP_LOGI` 就像自带智能排版的秘书，自动帮你附带了 3 个关键信息：

1. **I (颜色)**：在 VS Code 终端里直接显示为**醒目的绿色**，代表这是一条正常信息；如果是错误（`ESP_LOGE`）就会直接显示为**醒目的红色**！
2. **(1360)**：告诉你这是**开机后第 1360 毫秒**发生的，自带精准时间戳；
3. **LEVEL_1_HELLO**：告诉你这条信息是哪段代码打印出来的。

疑问 3：延时 1 秒，为什么不能写 `for(int i=0; i<1000000; i++)` 这种空循环？

这是所有初学者最容易踩的大坑！

- **❌ 空循环（死等发呆）**：
 - 相当于你为了等 1 分钟后的闹钟，在原地疯狂拼命做俯卧撑；
 - **后果**：CPU 满负荷运转、发烫发热，而且单片机内部有**看门狗（Watchdog）监视器**，一旦看门狗发现你在疯狂死转不理后台系统，就会判定“**程序死锁死机了**”，然后强行把单片机重启复位！
- **✅ `vTaskDelay()`（定闹钟睡觉）**：
 - 相当于你定了一个 1 秒后的闹钟，然后立刻躺下睡觉（把 CPU 算力让出来）；
 - 这 1 秒钟内芯片功耗极低，后台有其他任务也能顺畅运行，时间一到系统自动把你唤醒。

疑问 4： pdMS_TO_TICKS(1000) 到底是什么意思？

- 单片机操作系统（FreeRTOS）内部不以“毫秒”为单位，而是以***心跳节拍（Tick）***来计时的；
- 系统的时钟就像心跳一样，每秒跳 100 次（即 1 次心跳 = 10 毫秒）；
- pdMS_TO_TICKS(1000) 就是一个换算公式：

心跳次数 = 1000 毫秒 / 10 毫秒/次 = 100 次心跳

- 它的意思就是告诉操作系统：“请让我休眠 100 个心跳的时间”。

1.4 专属编程知识拓展（语法细节与库函数字典）

当你对上面的基本概念已经有感觉之后，这一节我们把代码中出现的**每一个头文件、每一个 C 语言语法细节**系统地梳理一遍，方便你以后随时翻阅查记！

1. 本章引入的头文件速查字典

引入的头文件	所属体系	提供的主要功能 / 函数 / 结构体	为什么本章需要它？
<stdio.h>	C 标准库	printf() 等标准输入输出函数	学习与对比标准 C 输出格式
<inttypes.h>	C99 标准库	PRIu32 格式化打印宏	跨芯片安全打印 32 位整型无警告
"freertos/FreeRTOS.h"	FreeRTOS 操作系统	FreeRTOS 底层核心类型定义与宏	任何使用操作系统 API 前的必备基础
"freertos/task.h"	FreeRTOS 操作系统	vTaskDelay() 任务休眠、pdMS_TO_TICKS()	实现让出 CPU 的高效无阻塞延时
"esp_log.h"	ESP-IDF 框架	ESP_LOGI()、ESP_LOGE()、ESP_LOGW()	打印带颜色、时间戳和模块标签的日志
"esp_system.h"	ESP-IDF 框架	esp_get_free_heap_size() 获取剩余内存	实时监控单片机内部 SRAM 运行内存
"esp_chip_info.h"	ESP-IDF 框架	esp_chip_info()、esp_chip_info_t 结构体	动态查询 CPU 核心数、芯片版本与无线特性
"esp_flash.h"	ESP-IDF 框架	esp_flash_get_size() 读取 Flash 容量	查询板载外部 Flash 存储器大小

2. 为什么在 main/CMakeLists.txt 里要写 REQUIRES ？

我们在编译时可能会遇到：fatal error: esp_flash.h: No such file or directory。在 main/CMakeLists.txt 中添加 spi_flash 后就成功了：

```
idf_component_register(
    SRCS "app_main.c"
    INCLUDE_DIRS "."
    REQUIRES driver esp_timer esp_psram spi_flash
)
```

• ESP-IDF 的“积木式”组件架构：

ESP-IDF 由几十个独立组件（驱动、定时器、Wi-Fi、Flash 等）拼装而成。为了加快编译速度并减少最终程序体积，**你的代码里用了哪个组件的头文件，就必须在 REQUIRES 后面显式声明该组件的名字。**

- 用到 gpio... → 需要 driver
- 用到 esp_flash... → 需要 spi_flash
- 用到 PSRAM 内存 → 需要 esp_psram

3. C 语言嵌入式语法细节剖析

① static const char *TAG = "LEVEL_1_HELLO";

- **const（常量）**：告诉单片机“这个字符串内容永不修改”。单片机就会直接把它放在大容量的 Flash（相当于电脑硬盘）里的只读区（.rodata），**不占用极其宝贵的 SRAM（运行内存）**！
- **static（静态私有）**：告诉单片机“这个 TAG 变量只在我当前这个 .c 文件里有效”。如果别人在其他文件也叫 TAG，两人不会冲突打架。

② 为什么传参要加取地址符 &？（指针初体验）

```
esp_chip_info_t chip_info;           // 1. 定义一个用于存放硬件信息的结构体变量
esp_chip_info(&chip_info);          // 2. 将该变量的内存地址 (&) 交给函数
```

- 在 C 语言中，函数传参默认是“复制一份副本”。如果直接传 chip_info，函数改的只是副本，改完就丢了；
- 加上 **&（取地址符）**，相当于把这个变量在内存里的“家庭住址”交给了 esp_chip_info 函数，函数就能顺着地址直接把查到的 CPU 核心数、版本号写到你的变量盒子里。

③ 位运算与特性检测：（chip_info.features & CHIP_FEATURE_WIFI_BGN）

- **位掩码（Bitmask）机制**：芯片有很多特性（Wi-Fi、BLE、经典蓝牙等），如果每个特性都用一个独立变量存很浪费空间；
- 乐鑫把所有特性压缩进一个整型数字的不同二进制位（Bit）里：
 - 第 0 位代表是否支持 Wi-Fi；
 - 第 4 位代表是否支持蓝牙；
- 使用**按位与 & 运算**，就能一瞬间判断出某一位是不是 1（即是否具备该硬件特性）。

④ 工业级错误码处理：esp_err_t 与 ESP_OK

```
if (esp_flash_get_size(NULL, &flash_size) == ESP_OK) {
    // 成功获取到了 Flash 大小
}
```

- ESP-IDF 中绝大部分函数都会返回一个状态码 esp_err_t；
- **ESP_OK（数值为 0）** 代表函数顺利执行成功；如果不等于 0 说明硬件读取失败了；
- 在写单片机代码时，**每次调用重要函数都检查返回值是否为 ESP_OK 是最重要的好习惯。**

1.5 动手小实验（新手即时正反馈）

学习编程最快的方法就是**亲手改改代码，看看会发生什么！**

请在 `main/app_main.c` 中尝试做以下 2 个小修改：

实验 1：把日志变成红色报警

把代码里的这一行：

```
ESP_LOGI(TAG, "[#%04d] Hello ESP32! 当前空闲内存: %" PRIu32 " 字节", count, free_heap);
```

改成 `ESP_LOGE`（E 代表 Error 错误）：

```
ESP_LOGE(TAG, "[#%04d] 警告！这是一条红色错误日志！", count);
```

 **重新烧录观察：**终端里的文字是不是瞬间变成鲜艳的**红色**了？

实验 2：让心跳加速 5 倍

把延时函数从 1000 毫秒改成 200 毫秒：

```
vTaskDelay(pdMS_TO_TICKS(200));
```

 **重新烧录观察：**终端打印的速度是不是像机关枪一样快速刷新了？

1.6 新手排错宝典（遇到问题看这里）

遇到的现象	为什么会这样？	怎么解决？
插上电脑没有任何反应，找不到 COM 端口	90% 是因为用了买充电宝送的“纯充电线”	换一根能给手机传照片的标准 Type-C 数据线
烧录固件时提示 <code>Timed out</code> 超时失败	单片机没能自动切换到下载状态	按住板子上的 BOOT 键 不放，按一下 RESET 键 松开，再松开 BOOT 键 ，然后重新点击烧录
串口监视器里输出一堆像乱码一样的问号	电脑接收的速度跟单片机发送的速度没对上	确认串口监视器右下角的波特率选的是 115200

1.7 本章总结与闯关打卡

恭喜你！学完本章，你已经迈出了单片机开发最坚实的第一步。你已经掌握了：

- 1. 单片机是如何通过串口（UART）向电脑汇报信息的；

2. 每一个头文件和关键 C 语言语法（`static`、`const`、`&` 取地址、位运算）的物理含义；
3. 为什么嵌入式程序必须在 `while(1)` 中配合 `vTaskDelay()` 休眠运行；
4. 如何像专业工程师一样使用 `ESP_LOG` 打印漂亮的彩色调试日志。

下一关预告：既然代码已经能在电脑屏幕上打字了，那么下一关，我们将控制电流流出单片机，去**点亮开发板上的第一颗硬件蓝色 LED 灯 (GPIO 输出与呼吸灯) !**